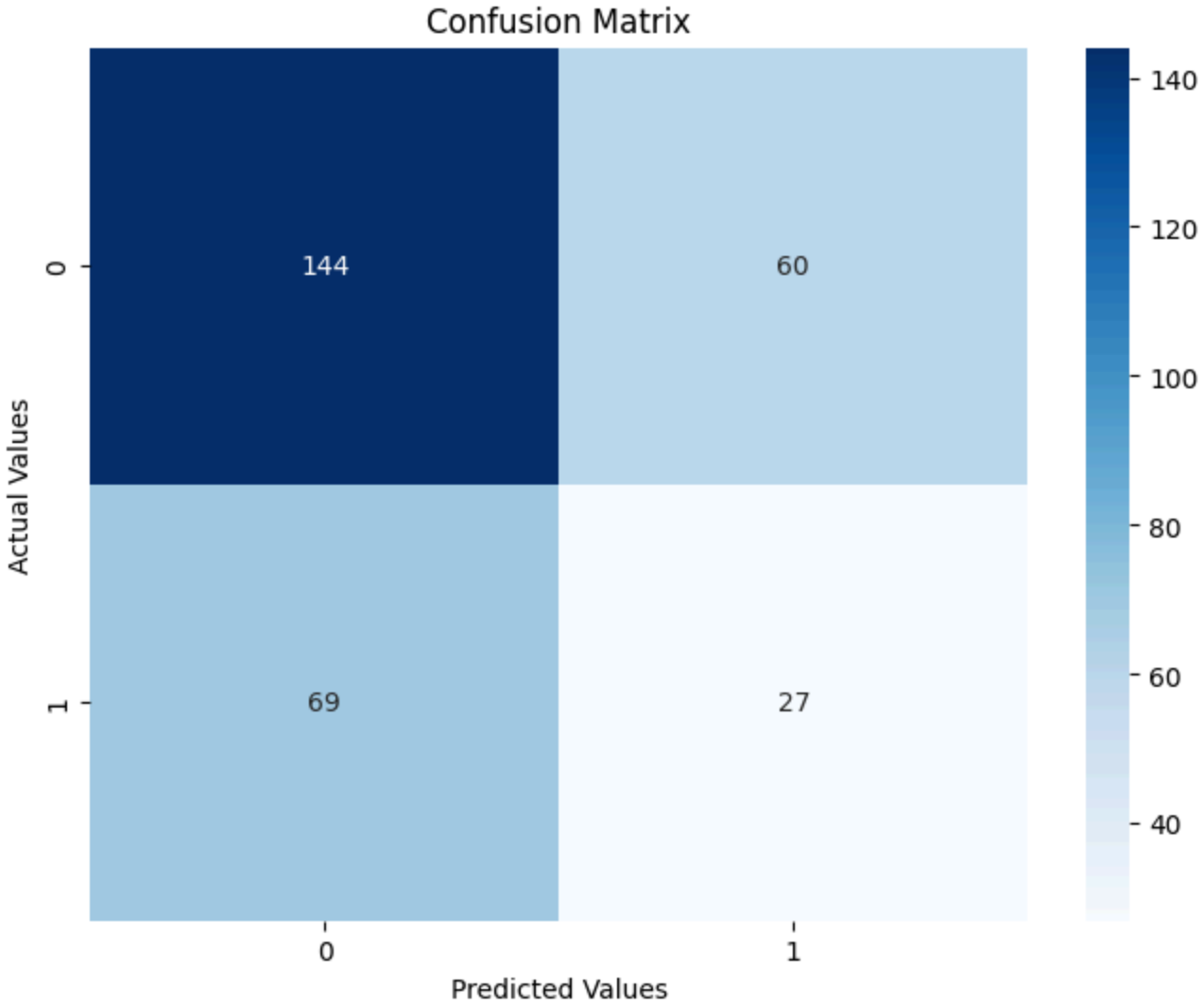
Step-by-Step Instructions:

In [1]:

```
# Hello everyone, in this lab, we'll walk through a Decision Tree Classification example using a customer churn dataset
# We'll use pandas for data manipulation, matplotlib and seaborn for plotting, and sklearn for the Decision Tree model

import pandas as pd  # Importing pandas for data manipulation
import numpy as np  # Importing numpy for numerical operations
from sklearn.model_selection import train_test_split  # Importing train_test_split from sklearn for splitting the data
from sklearn.tree import DecisionTreeClassifier  # Importing DecisionTreeClassifier from sklearn
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix  # Importing evaluation met
import matplotlib.pyplot as plt  # Importing matplotlib for plotting
import seaborn as sns  # Importing seaborn for advanced visualizations
import warnings  # Importing warnings to manage warning messages

# We'll ignore any warnings to keep our output clean.
warnings.filterwarnings('ignore')  # This will suppress any warning messages

# Let's start by loading our dataset into a pandas DataFrame.
file_path = 'churn_data.csv'  # Path to the CSV file
data = pd.read_csv(file_path)  # Reading the CSV file into a DataFrame

# Displaying the first 5 rows of the dataset to get an idea of what it looks like.
print(data.head())  # Displaying the first 5 rows of the DataFrame

# Separating the independent variables (features) and dependent variable (target).
X = data.drop('Churn', axis=1)  # Dropping the 'Churn' column from the features
y = data['Churn']  # Selecting the 'Churn' column as the target variable

# Convert categorical variables into dummy/indicator variables.
X = pd.get_dummies(X)

# Splitting the data into training and testing sets.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)  # Splitting the data with 70

# Creating a Decision Tree model and fitting it to the training data.
model = DecisionTreeClassifier(random_state=42)  # Creating an instance of DecisionTreeClassifier
model.fit(X_train, y_train)  # Fitting the model to the training data

# Using the model to predict churn on the test data.
y_pred = model.predict(X_test)  # Making predictions on the test data

# Calculating and printing the accuracy, precision, and recall of the model's predictions.
accuracy = accuracy_score(y_test, y_pred)  # Calculating accuracy
precision = precision_score(y_test, y_pred, pos_label='Yes')  # Calculating precision
```

```python
recall = recall_score(y_test, y_pred, pos_label='Yes')  # Calculating recall
print(f"Accuracy: {accuracy:.2f}")  # Printing accuracy
print(f"Precision: {precision:.2f}")  # Printing precision
print(f"Recall: {recall:.2f}")  # Printing recall

# Calculating the confusion matrix.
conf_matrix = confusion_matrix(y_test, y_pred)  # Calculating the confusion matrix

# Plotting the confusion matrix as a heatmap.
plt.figure(figsize=(8, 6))  # Creating a new figure with a specified size (8 inches by 6 inches)
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap='Blues')  # Creating a heatmap with the confusion matrix
plt.title('Confusion Matrix')  # Adding a title to the heatmap
plt.ylabel('Actual Values')  # Labeling the y-axis
plt.xlabel('Predicted Values')  # Labeling the x-axis
plt.show()  # Displaying the heatmap


#Note that if we'd like to develop a random forest classifier all we have to do is import then use RandomForestClassifi
```

```
   Unnamed: 0  MonthlyCharges        Contract  ServiceUsage  \
0           0       79.393215        One year      9.246997
1           1       94.367043        One year     10.134628
2           2       84.248704        Two year      3.111158
3           3       79.039486  Month-to-month      0.991352
4           4       68.128932        Two year      0.113940

   CustomerSupportCalls Churn
0                    10   Yes
1                     9    No
2                     8    No
3                     0    No
4                     0    No
Accuracy: 0.57
Precision: 0.31
Recall: 0.28
```



Let's take a look at the flow we followed for solving this problem and let's take a look at DT hyperparameters

Import Libraries:

We begin by importing necessary libraries including pandas for data manipulation, numpy for numerical operations, matplotlib and seaborn for plotting, and sklearn for machine learning tasks. We also import warnings to suppress any unwanted warnings. Load and Inspect Data:

We load the customer churn dataset into a DataFrame using pandas. We inspect the first few rows of the data to understand its structure and contents.

Prepare Data:

We separate the independent variables (features) and the dependent variable (target) from the dataset. We convert categorical variables into dummy/indicator variables for the model to process.

Split Data:

We split the data into training and testing sets using train_test_split. Here, 70% of the data is used for training the model, and 30% is used for testing.

Train Model:

We create an instance of the DecisionTreeClassifier model. We fit the model to the training data using the fit method.

Make Predictions:

We use the trained model to make predictions on the test data using the predict method.

Evaluate Model:

We evaluate the model's performance by calculating accuracy, precision, and recall scores. We also calculate the confusion matrix to summarize prediction results.

Visualize Results:

We visualize the confusion matrix using a heatmap for better understanding and interpretation. The heatmap displays the actual vs. predicted values, helping us see where the model is performing well and where it may be making mistakes.

Machine Learning Hyperparameters

random_state: This parameter ensures reproducibility by controlling the random number generation. Setting it to a fixed value ensures that the results can be replicated. max_depth: Limits the maximum depth of the tree. Deeper trees can model more complex patterns but are prone to overfitting. min_samples_split: The minimum number of samples required to split an internal node. Higher values prevent the model from learning overly specific patterns. min_samples_leaf: The minimum number of samples required to be at a leaf node. Helps in smoothing the model, especially in regression. criterion: This parameter specifies the function to measure the quality of a split. Common options are "gini" for the Gini impurity and "entropy" for the information gain. Understanding these hyperparameters allows us to fine-tune the model for better performance and to avoid overfitting or underfitting.

In [13]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [3]:
```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

Out[3]:

```
▾ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

In [4]:

```
rf_pred = rf_model.predict(X_test)
```

In [5]:

```
rf_accuracy = accuracy_score(y_test, rf_pred)
rf_precision = precision_score(y_test, rf_pred, pos_label='Yes')
rf_recall = recall_score(y_test, rf_pred, pos_label='Yes')
```

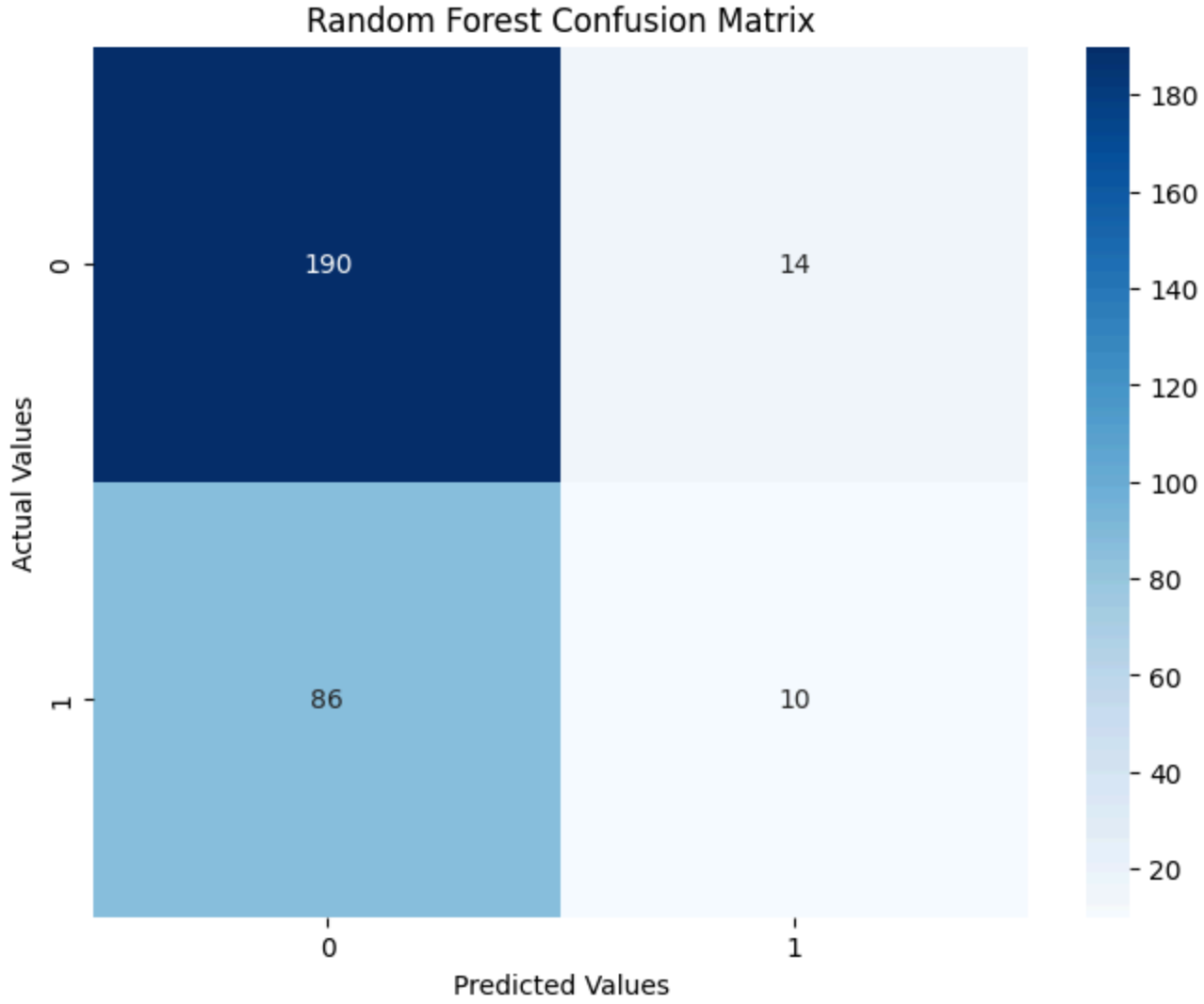
In [6]:

```
print(f"Random Forest Accuracy: {rf_accuracy:.2f}")
print(f"Random Forest Precision: {rf_precision:.2f}")
print(f"Random Forest Recall: {rf_recall:.2f}")
```

```
Random Forest Accuracy: 0.67
Random Forest Precision: 0.42
Random Forest Recall: 0.10
```

In [10]:

```
rf_conf_matrix = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(rf_conf_matrix, annot=True, fmt="d", cmap='Blues')
plt.title('Random Forest Confusion Matrix')
plt.ylabel('Actual Values')
plt.xlabel('Predicted Values')
plt.show()
```


Random Forest Confusion Matrix
190
14
86
10
0
1
Actual Values
0
1
Predicted Values
180
160
140
120
100
80
60
40
20

In [ ]: